

PROJECT REPORT

Cab Booking

Project Report

Team Lead: Paidipati Manikumar

Date: 15 July 2026

github.com/manikumar26092005/Cab-Booking-Premium

1. Introduction

1.1 Project Overview

Cab Booking is a full-stack MERN (MongoDB, Express.js, React, Node.js) cab booking web application built as part of a Full Stack Development internship. It provides three role-based portals — Rider, Driver, and Admin — connected through a shared REST API, allowing riders to book cabs, drivers to manage assigned rides and earnings, and admins to oversee users, drivers, cars, bookings, and platform analytics.

1.2 Purpose

The purpose of Cab Booking is to replace informal, phone-based cab dispatch with a centralized digital platform that gives riders transparent pricing and real-time booking visibility, gives drivers a structured and fair stream of ride requests with clear earnings tracking, and gives platform operators a single dashboard to manage the entire fleet and monitor business performance.

2. Ideation Phase

2.1 Problem Statement

Riders: I am a daily urban commuter trying to book a reliable, affordable cab quickly, but existing options have unpredictable wait times and unclear pricing, because drivers are assigned manually with no real-time visibility — which makes me feel frustrated and anxious about being late.

Drivers: I am a part-time cab driver trying to receive consistent ride requests and track my earnings, but I have no structured system to manage bookings and routes, because dispatch is handled informally over phone calls — which makes me feel disorganized and undervalued.

2.2 Empathy Map Canvas

Persona: Urban Commuter (Rider). Think & Feel: worried about on-time arrival and fair pricing. See: crowded traffic, multiple competing apps. Hear: friends complaining about cancellations. Say & Do: compares fares, calls drivers directly. Pain: unreliable ETAs, hidden charges. Gain: transparent pricing, real-time status, a trustworthy rated driver.

2.3 Brainstorming

The team brainstormed and grouped ideas under four themes — Access & Security, Booking & Transparency, Trust & Feedback, and Administration & Insights — then prioritized them on an Importance–Feasibility grid. Role-based portals, JWT authentication, and real-time booking status tracking scored highest on both axes and were selected as the core of the Cab Booking MVP, ahead of lower-priority ideas like in-app chat and advance scheduling.

3. Requirement Analysis

3.1 Customer Journey Map

Mapped across Entice → Enter → Engage → Exit → Extend stages for a rider booking a cab: from discovering Cab Booking, through registering and requesting a ride, to tracking the trip in real time, receiving a PDF receipt, and finally rating the driver. Key opportunity areas identified included live driver-location display during 'On the Way' status and prompting riders immediately after trip completion to improve review rates.

3.2 Solution Requirement

Functional requirements span User Registration, Authentication, Booking Management, Driver Assignment, Fare & Payment, Ratings & Reviews, and Admin Analytics. Non-functional requirements cover Usability (role-specific responsive dashboards), Security (JWT, RBAC, hashed passwords), Reliability (a consistent booking state machine), Performance (indexed MongoDB queries), Availability (continuous Express/MongoDB operation), and Scalability (modular MVC design ready to move to cloud hosting).

3.3 Data Flow Diagram

At Level 0, data flows between three external entities — Rider, Driver, and Admin — and four core processes: Manage Booking & Assignment, Process Trip & Fare, Generate Receipt & Notify, and Analytics & User Management, backed by Users/Drivers, Bookings, Payments, and Reviews data stores.

3.4 Technology Stack

Layer	Technology
Frontend	React (Vite), HTML, CSS, JavaScript
Backend	Node.js, Express.js (MVC architecture)
Database	MongoDB (Community Server, local)
Authentication	JSON Web Tokens (JWT), bcrypt password hashing
Receipts	PDFKit (PDF generation)
Analytics	Recharts (React charting library)
Notifications	Email service (registration/confirmation)

4. Project Design

4.1 Problem Solution Fit

Urban commuters and drivers (customer segments) need a trustworthy, organized booking system (job-to-be-done), currently unmet due to manual, informal dispatch (root cause). Cab Booking's role-based portals, real-time booking lifecycle, and transparent fare display directly close this gap, shifting the emotional experience from pre-booking anxiety to post-booking confidence.

4.2 Proposed Solution

S.No.	Parameter	Description
1.	Problem Statement	Riders lack reliable, transparent cab booking; drivers lack structured, fair access to ride requests.
2.	Idea / Solution	A MERN-stack platform with Rider, Driver, and Admin portals, JWT auth, real-time booking lifecycle, fare estimation, PDF receipts, and admin analytics.
3.	Novelty	Unified real-time visibility of the same booking lifecycle across all three stakeholders, with built-in receipts and analytics.
4.	Social Impact	Reduces rider anxiety and gives drivers a fair, organized income stream and trip record.
5.	Business Model	Commission-based revenue on completed rides.
6.	Scalability	Modular MVC backend and MongoDB data layer ready to scale to cloud hosting as usage grows.

4.3 Solution Architecture

A three-tier architecture: a React (Vite) client for Rider/Driver/Admin portals talks to an Express.js REST API (35 endpoints) guarded by JWT/RBAC middleware; MVC controllers handle Booking, User, Driver, Payment, and Review logic against a MongoDB database (5 collections), while PDFKit generates trip receipts, Recharts powers the admin analytics dashboard, and an email service handles registration/confirmation notifications.

5. Project Planning & Scheduling

5.1 Project Planning

Development was organized into 4 sprints (Sprint-1 through Sprint-4) covering 15 user stories (USN-1 to USN-15) across Registration, Login, Role Management, Booking, Driver Assignment, Driver Actions, Payment, Feedback, Admin Dashboard, and Security epics, totalling 36 story points. All four sprints completed on their planned end dates (06 Jun, 13 Jun, 20 Jun, and 27 Jun 2026 respectively) with no carry-over of incomplete stories, reflecting a steady, predictable delivery pace.

6. Functional and Performance Testing

6.1 Performance Testing

Functional tests validated input validation, booking creation, and JWT/role-based access control across all endpoints. Performance tests measured booking API response time (~0.8 seconds average, against a 2-second target), concurrent booking load (handled correctly with database-level assignment locking and no duplicate driver assignments), PDF receipt generation under load (no errors across rapid successive requests), and admin analytics dashboard load time (~1.5 seconds against a 3-second target). Full User Acceptance Testing covered 59 test cases across 7 modules, all passing, with 27 logged defects triaged and closed.

7. Results

7.1 Output Screenshots

Sign-in screen (shared by Rider, Driver, and Admin):

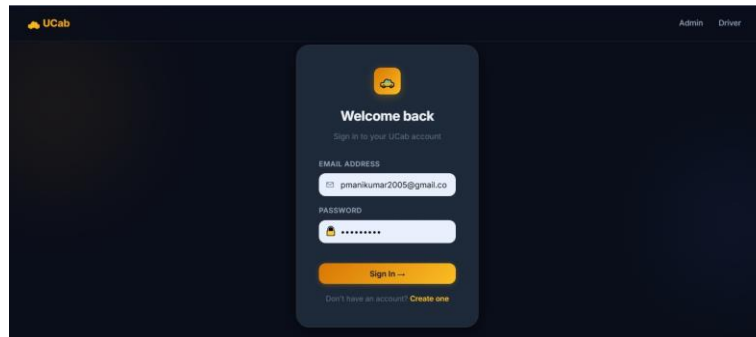


Figure 7.1: UCab sign-in screen

Rider — Book a Ride with fare estimation:

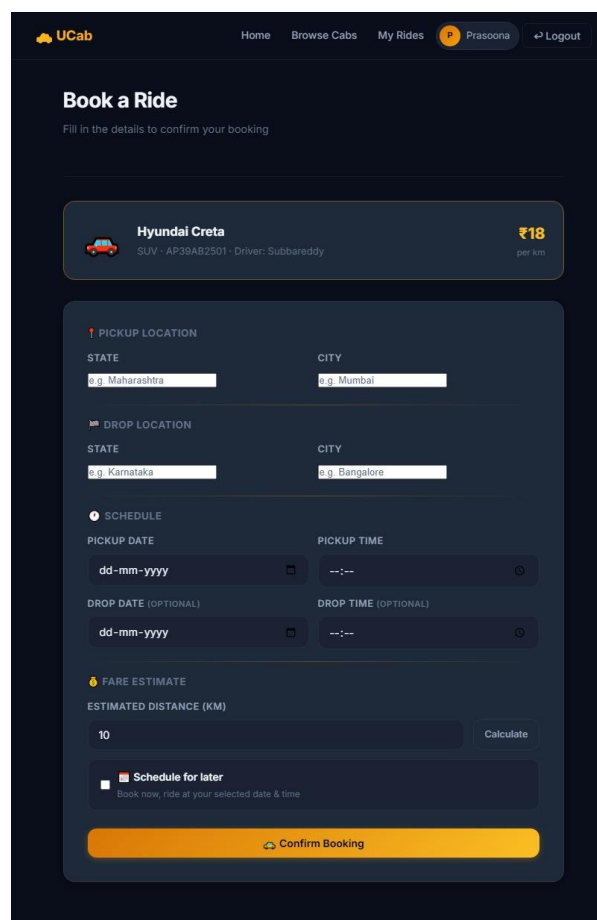


Figure 7.2: Rider booking flow with pickup/drop details and fare estimate

Rider — My Rides history with completed trips:

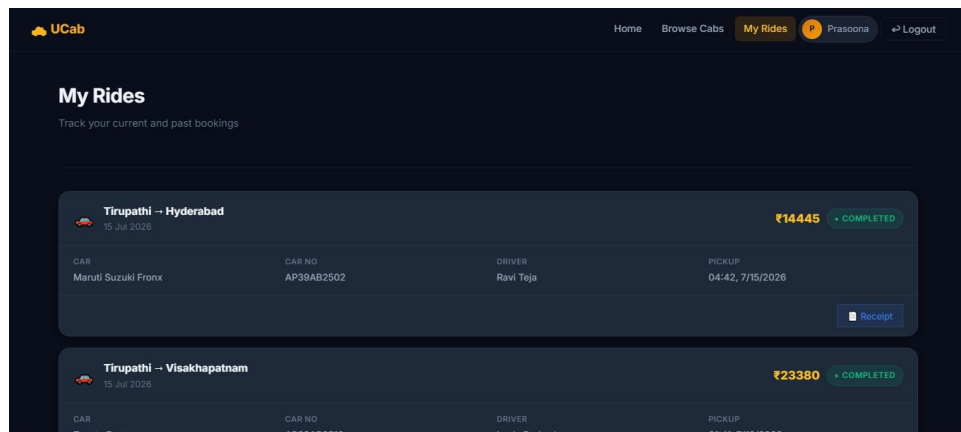


Figure 7.3: Rider ride history with receipt access

Driver Dashboard — earnings and assigned rides:

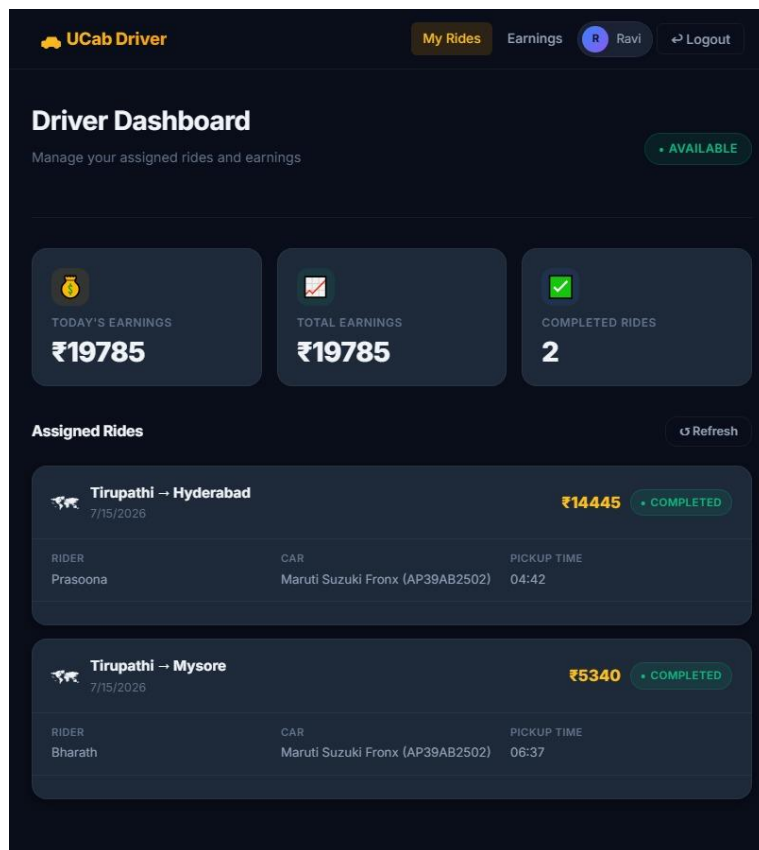


Figure 7.4: Driver dashboard with today's/total earnings and completed rides

Driver — assigned rides detail view:

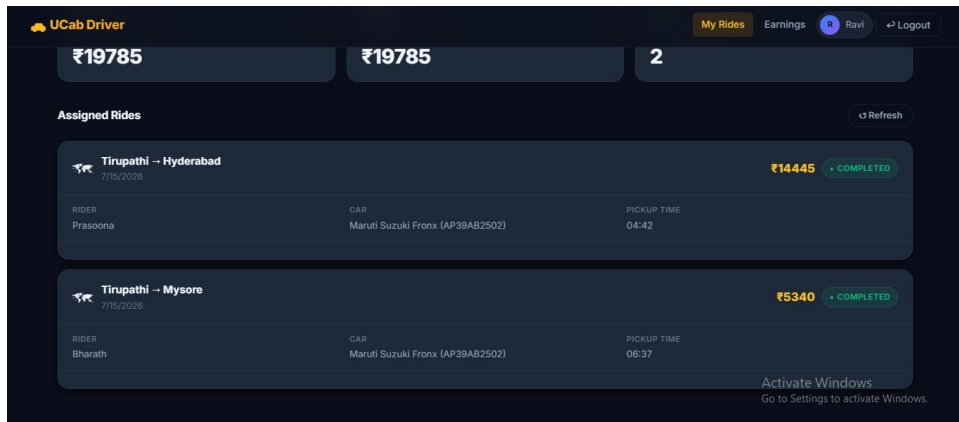


Figure 7.5: Driver's assigned rides with rider, car, and pickup-time details

Admin — Platform Overview dashboard:

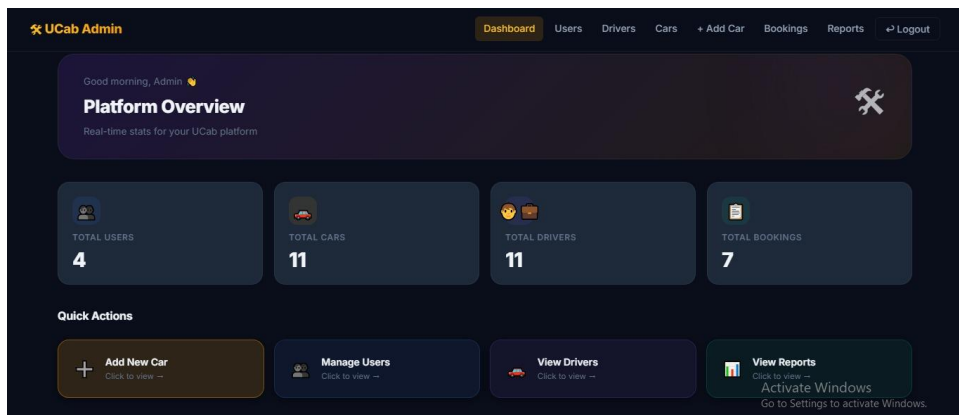


Figure 7.6: Admin dashboard with total users, cars, drivers, and bookings

Admin — Drivers management:

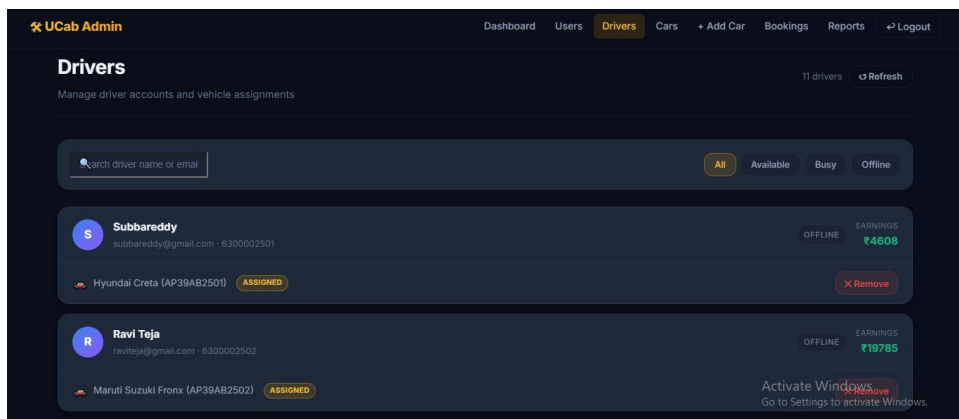


Figure 7.7: Admin drivers list with availability status, vehicle assignment, and earnings

Admin — Bookings management with live status tracking:

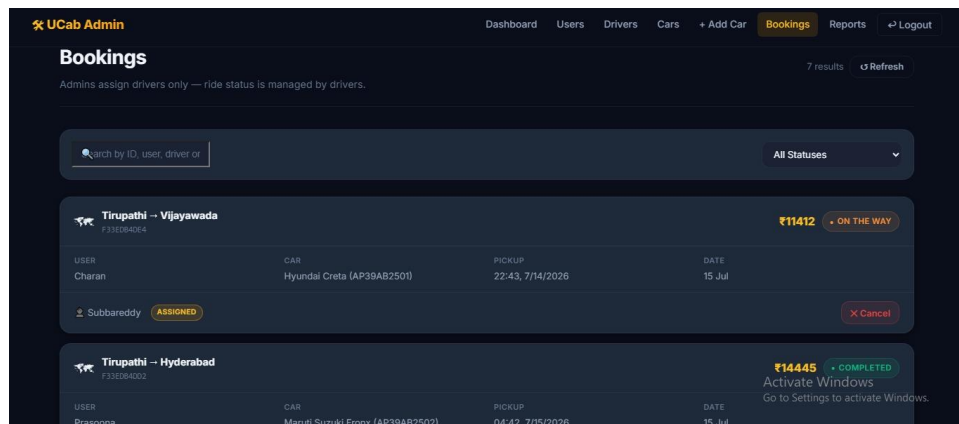


Figure 7.8: Admin bookings list with status filter and per-ride details

8. Advantages & Disadvantages

Advantages

- Centralized, transparent booking lifecycle visible to rider, driver, and admin alike
- Role-based security (JWT + RBAC) protects sensitive admin and driver operations
- Automated fare estimation and PDF receipts remove manual, error-prone calculations
- Admin analytics dashboard gives at-a-glance visibility into platform health
- Modular MVC backend makes the system straightforward to extend and maintain

Disadvantages

- Currently runs on a local MongoDB instance rather than a cloud-hosted, highly available database
- No integrated online payment gateway yet — fare is calculated and receipted, but not collected in-app
- No live map-based GPS tracking of drivers during a trip
- No in-app chat/call bridge between rider and driver

9. Conclusion

Cab Booking successfully demonstrates a complete, role-based cab booking platform built on the MERN stack, covering the full journey from ideation and requirement analysis through design, planning, development, and testing. The application meets its core goal of replacing manual, informal dispatch with a transparent, trackable booking lifecycle for riders, drivers, and administrators, validated by a full round of functional, performance, and user acceptance testing with all test cases passing.

10. Future Scope

- Live map-based driver tracking during active trips

- Online payment gateway integration for in-app fare collection
- In-app chat or call bridge between rider and driver
- Migration from local MongoDB to MongoDB Atlas and cloud deployment for production scalability
- Push notifications for booking status changes
- Refined scheduled/advance ride booking with reminders

11. Appendix

Source Code: Included in the linked GitHub repository (client and server directories).

Dataset Link: Not applicable — Cab Booking uses live application data (Users, Drivers, Cars, Bookings) stored in MongoDB rather than a static dataset.

GitHub Repository: <https://github.com/manikumar26092005/Cab-Booking-Premium>

Project Demo: <https://drive.google.com/drive/folders/1ZDpZ5FvcYIxTEkNpBHU6LG2rhIlf5tR>

Team

Name	Email	Role
Paidipati Manikumar	pmanikumar2005@gmail.com	Team Lead
Godugunuru Penchala Raviteja	godugunuruteja@gmail.com	Member
Puli Mokshagna	mokshapuli7@gmail.com	Member
Panthagani Rupan Raj	panthaganiirupanraj@gmail.com	Member
Sai Ganesh Uday Kiran Bokkasam	udaykiranroyals9099@gmail.com	Member